

Cluster, Worker Threads & Worker Pool in Node.js

1. Cluster in Node.js

- 1 Cluster allows Node.js to utilize multiple CPU cores by creating child processes (workers).
- 2 Each worker has its own event loop, memory, and handles requests independently.
- 3 Requests are distributed among workers by the master process.
- 4 Advantage: Utilizes full CPU cores for better concurrency.
- 5 Disadvantage: Workers do not share memory directly, only communicate via IPC.

2. Worker Threads

- 1 Worker Threads allow running JavaScript in parallel using threads within a single process.
- 2 Useful for CPU-intensive tasks like encryption, image processing, etc.
- 3 Threads share memory via SharedArrayBuffer.
- 4 Advantage: Less memory overhead compared to cluster.
- 5 Disadvantage: Thread creation overhead + context switching increases delay if too many threads are created.

3. Worker Pool

- 1 Worker Pool is a pool of pre-created worker threads reused for multiple tasks.
- 2 Tasks are queued, and idle workers pick tasks when available.
- 3 Advantage: Avoids thread creation overhead, reduces context switching delays.
- 4 Advantage: Handles large number of requests more efficiently.
- 5 Disadvantage: If pool size is too small, tasks may wait in queue (latency).

4. Example: Handling 300 Requests on 8-core CPU

- 1 Cluster: 8 workers → each handles ~38 requests independently.
- 2 Worker Threads: If 32 threads created per worker → high thread creation cost + context switching delays.
- 3 Worker Pool: 8 pre-created worker threads → tasks are queued and executed efficiently.
- 4 Result: Worker Pool performs faster because it avoids repeated thread creation overhead.

5. Role of Libuv

- 1 Libuv is the library that powers Node.js asynchronous I/O.
- 2 Manages the event loop and delegates I/O operations to OS threads.
- 3 For CPU-intensive tasks: worker threads or pools are needed.
- 4 Cluster + Worker Threads → utilize multiple processes and threads for scaling.

6. Conclusion

- 1 Cluster is best for scaling across CPU cores.

- 2 Worker Threads are good for parallel CPU tasks, but too many threads slow performance.
- 3 Worker Pool provides best efficiency by reusing threads.
- 4 Choose based on workload: Cluster for I/O-heavy, Worker Pool for CPU-heavy.